NumPy

Tính trên cả mảng thay cho vòng lặp

Phase 0 · Bài 0.2

Một dòng thay hai vòng lặp

40 học sinh, 3 môn. Chuẩn hóa điểm từng môn:

```
Z = (X - X.mean(axis=0)) / X.std(axis=0)
```

Ba ý tưởng: **shape**, **axis**, **broadcasting**.

CÂU HỎI

Ảnh màu là $(H, W, 3)$. Batch ảnh là $(N, 3, H, W)$.

Trung bình từng kênh màu của cả batch có shape gì?

Sau buổi này, các em có thể

- Viết lại vòng lặp thành phép toán mảng, đo được chênh lệch tốc độ.
- Đoán trước shape khi gộp theo axis và khi broadcasting.
- Phân biệt view và copy.
- Tránh lỗi $(n,)$ trừ $(n, 1)$, tràn số `uint8`, lỗi shape âm thầm.

01

Mảng và kiểu dữ liệu

shape, dtype, reshape

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape, a.ndim, a.size, a.dtype)
print(np.arange(12).reshape(3, -1).shape)
```

KẾT QUẢ

```
(2, 3) 2 6 int64
(3, 4)
```

- Một khối số liền nhau, **cùng một kiểu**.
- `reshape` chỉ đổi cách đọc; `-1` để NumPy tự tính.
- Học máy: `float32` trên GPU, `float64` trên CPU.

Tràn số không báo lỗi

```
print(np.array([2**62]) * 4)
pixels = np.array([200, 100], dtype=np.uint8)
print(pixels + 100)
x = np.array([1, 2, 3]); x[0] = 2.7
```

KẾT QUẢ

```
[0]
[ 44 200]
[2 2 3]
```

- 2^{64} quay vòng về 0.
- Ảnh `uint8`: $300 \to 44$. Đổi sang `float32` trước khi tính.
- Gán số thực vào mảng nguyên: bị cắt.

HỎI NHANH

`np.array([250, 10], dtype=np.uint8) + 10` cho kết quả gì?

A `[260, 20]`

B `[4, 20]`

C `[255, 20]`

D Báo lỗi tràn số

02

Tính trên cả mảng

Vector hóa

```
z = np.array([-2.0, 0.0, 1.0, 3.0])
print(1 / (1 + np.exp(-z)))
print(z > 0, (z > 0).sum(), (z > 0).mean())
print(np.where(z > 0, z, 0.0))
```

KẾT QUẢ

```
[0.11920292 0.5        0.73105858 0.95257413]
[False False  True  True] 2 0.5
[0. 0. 1. 3.]
```

- So sánh cho mảng `bool`; `True` tính là 1.
- Accuracy: `(pred == y).mean()`.
- Sigmoid 10^6 số: vòng lặp 0.14 giây, NumPy 0.005 giây.

View và copy

```
a = np.arange(6)
s = a[1:4]; s[0] = 100        # view
f = a[[1, 2, 3]]; f[0] = -1   # bản sao
print(a)
```

KẾT QUẢ

```
[  0 100   2   3   4   5]
```

Cách lấy	Kết quả
`a[1:4]`, `a[:, 0]`	**view**: sửa là đổi gốc
`a[[1, 2]]`, `a[a > 0]`	bản sao

Khác list Python: lát cắt list là bản sao. Cần tách thì `.copy()`.

HỎI NHANH

`a = np.zeros(5), b = a[2:], b += 1.` a là gì?

A `[0, 0, 0, 0, 0]`

B `[0, 0, 1, 1, 1]`

C `[1, 1, 1, 1, 1]`

D `[0, 0, 1]`

03

axis

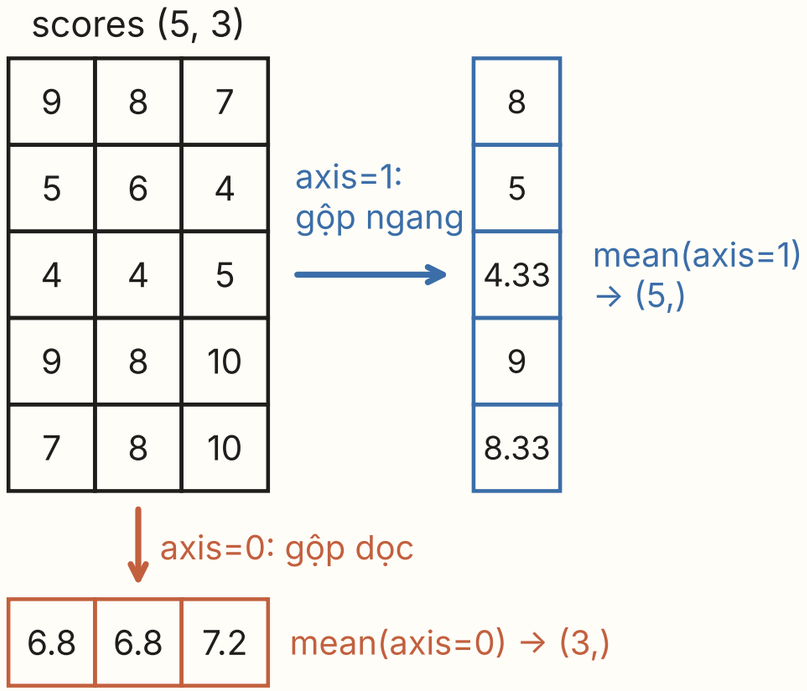
Trục nào ghi trong `axis` thì biến mất

```
scores.shape                    # (5 học sinh, 3 môn)
print(scores.mean(axis=0))
print(scores.mean(axis=1).round(2))
```

KẾT QUẢ

```
[6.8 6.8 7.2]
[8.   5.   4.33 9.   8.33]
```

Quy ước: **trục 0 là các mẫu**.



axis=0 gộp dọc, axis=1 gộp ngang.

Mảng nhiều chiều

```
imgs = rng.random((100, 28, 28))
print(imgs.mean(axis=0).shape)
print(imgs.mean(axis=(1, 2)).shape)
print(imgs.mean(axis=(1, 2), keepdims=True).shape)
```

KẾT QUẢ

```
(28, 28)
(100,)
(100, 1, 1)
```

- Ảnh trung bình: gộp trục ảnh.
- Độ sáng từng ảnh: gộp hai trục không gian.
- `keepdims=True` giữ trục bị gộp với độ dài 1.

HỎI NHANH

`X` có shape $(32, 3, 64, 64)$. `X.mean(axis=(0, 2, 3))` có shape gì?

A $(32,)$

B $(3,)$

C $(64, 64)$

D $(32, 3)$

04

Broadcasting

Hai quy tắc

1. Xếp shape **từ phải**; thiếu trục thì thêm trục độ dài 1 bên trái.
2. Mỗi trục: bằng nhau, hoặc một bên bằng 1 (được kéo dãn).

A	B	Kết quả
$(5, 3)$	$(3,)$	$(5, 3)$
$(5, 3)$	$(5, 1)$	$(5, 3)$
$(4, 1)$	$(3,)$	$(4, 3)$
$(5, 3)$	$(5,)$	**Lỗi**

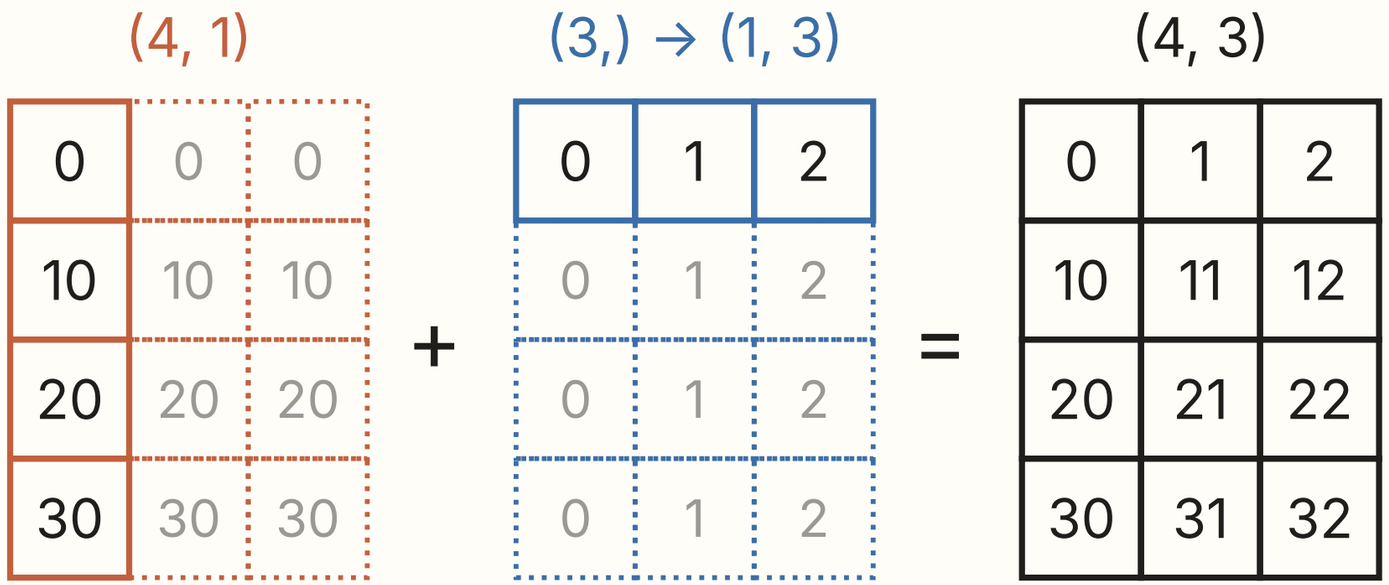
```
mu = scores.mean(axis=0)        # (3,)
Z = (scores - mu) / scores.std(axis=0)
print(Z.shape)
```

KẾT QUẢ

```
(5, 3)
```

`mu` được kéo dãn cho mỗi hàng, không tạo bản sao thật.

Kéo dãn trục độ dài 1



xếp shape từ phải:

`(4, 1)`

`(1, 3)`

→ `(4, 3)`

ô nét chấm: phần kéo dãn

(4, 1) + (3,): cột lặp sang ngang, hàng lặp xuống dọc, thành (4, 3).

keepdims cứu phép trừ theo hàng

```
scores - scores.mean(axis=1)
```

KẾT QUẢ

```
ValueError: operands could not be broadcast together with shapes (5,3) (5,)
```

```
scores - scores.mean(axis=1, keepdims=True)   # (5, 1)
```

Khoảng cách mọi cặp điểm, thay hai vòng lặp:

```
diff = A[:, None, :] - B[None, :, :]
D = np.sqrt((diff ** 2).sum(axis=-1))
```

$(500, 300, 64)$: 9.6 triệu số, 77 MB. Với $n = m = 10^4$: 51 GB.

HỎI NHANH

`A` có shape $(8, 1, 6)$, `B` có shape $(7, 1)$. `(A + B).shape`?

A Lỗi

B $(8, 7, 1)$

C $(8, 7, 6)$

D $(8, 1, 6)$

$(n,)$ và $(n, 1)$

```
y = np.array([1.0, 2.0, 3.0])           # (3,)
y_hat = np.array([[1.0], [2.0], [3.0]])  # (3, 1), đúng hoàn toàn
print((y_hat - y).shape)
print(np.mean((y_hat - y) ** 2))
```

KẾT QUẢ

```
(3, 3)
1.3333333333333333
```

- Đoán đúng hết mà MSE $=$ 1.33, không báo lỗi.
- $(3, 1) - (3,)$ thành ma trận mọi hiệu $\hat{y}_i - y_j$.
- Phòng: `assert y_hat.shape == y.shape`, dùng `.ravel()`.

05

Đại số tuyến tính và số ngẫu nhiên

@ khác *

```
M = np.array([[2.0, 1.0], [1.0, 3.0]])
v = np.array([1.0, 2.0])
print(M @ v, np.linalg.solve(M, v))
```

KẾT QUẢ

```
[4. 7.] [0.2 0.6]
```

$(n, k)@(k, m) \to (n, m)$. $*$ nhân từng phần tử.

```
rng = np.random.default_rng(42)
print(rng.integers(0, 10, size=5))
rng2 = np.random.default_rng(42)
print(rng2.integers(0, 10, size=5))
```

KẾT QUẢ

```
[0 7 6 4 4]
[0 7 6 4 4]
```

Cùng seed, cùng dãy số.

Tính tay trên lớp

1. `np.arange(3).reshape(3, 1) + np.arange(3)`
2. `np.ones((2, 3)).sum(axis=0)`
3. `np.ones((4, 5, 6)).sum(axis=(0, 2), keepdims=True).shape`
4. `np.array([[1, 2], [3, 4]]) @ np.array([1, 1])`

1. $(3, 3)$: `[[0, 1, 2], [1, 2, 3], [2, 3, 4]]`
2. `[2, 2, 2]`
3. $(1, 5, 1)$, mỗi phần tử bằng 24
4. `[3, 7]`

Bẫy cài đặt

Bẫy	Hậu quả	Cách tránh
`if a > 0:` với mảng	`ValueError: ... ambiguous`	`.all()` hoặc `.any()`
Sửa view	Mảng gốc đổi theo	`.copy()`
Cộng trừ `uint8`	Quay vòng	Đổi `float32` trước
$(n,)$ với $(n, 1)$	Số sai, không báo lỗi	`assert` shape, `.ravel()`
Broadcasting quá lớn	Hết bộ nhớ	Ước lượng: 8 byte mỗi số

Khi đi thi

- **In shape ở mọi bước**: phần lớn lỗi về sau là lỗi shape.
- **Không lặp trên từng mẫu**; vòng lặp lồng nhau thì nghĩ đến broadcasting hoặc `@`.
- **Kiểm tra** `dtype` khi đọc ảnh, dữ liệu.
- **Một** `rng` **có seed** cho cả notebook.
- **Ước lượng bộ nhớ** trước khi tạo mảng lớn.

Tóm tắt

- Mảng có `shape`, `dtype` cố định; số nguyên có thể tràn âm thầm.
- Phép toán trên cả mảng nhanh hơn vòng lặp Python hàng chục lần.
- Lát cắt là view; list chỉ số, mặt nạ `bool` là bản sao.
- `axis` nào thì trục đó biến mất; `keepdims` giữ lại độ dài 1.
- Broadcasting xếp từ phải, kéo dãn trục độ dài 1; cẩn thận $(n,)$ và $(n, 1)$.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook của bài: viết lại `pairwise_dist` bằng công thức $\|a-b\|^2 = \|a\|^2 + \|b\|^2 - 2a^\top b$, so tốc độ và bộ nhớ với bản broadcasting.